arXiv:2509.00195v2 [cs.LG] 31 Jan 2026

FastTTS: Accelerating Test-Time Scaling for Edge LLM Reasoning

Hao Mark Chen
Imperial College London
London, UK
hao.chen20@imperial.ac.uk

Zhiwen Mo
Imperial College London
London, UK
zhiwen.mo25@imperial.ac.uk

Guanxi Lu
Imperial College London
London, UK
guanxi.lu22@imperial.ac.uk

Shuang Liang
Imperial College London
London, UK
shuang.liang@imperial.ac.uk

Lingxiao Ma
Microsoft Research
Beijing, China
lingxiao.ma@microsoft.com

Wayne Luk
Imperial College London
London, UK
w.luk@imperial.ac.uk

Hongxiang Fan
Imperial College London
London, UK
hongxiang.fan@imperial.ac.uk

Abstract

Recent advances in reasoning Large Language Models (LLMs) are driving the emergence of agentic AI systems. Edge deployment of LLM agents near end users is increasingly necessary to protect data privacy, enable offline use, and provide responsive interaction with local context. However, strict memory constraints on edge devices limit deployment to smaller LLMs, whose reasoning capabilities are much weaker than those of large cloud models, hindering practical deployment of edge agentic AI. Test-Time Scaling (TTS) offers a promising solution by allocating more compute during inference to enhance the reasoning capability of edge LLMs. However, current TTS methods introduce heavy hardware performance overhead on resource-constrained devices, making them impractical for real applications. To address this challenge, we present *FastTTS*, a serving system that enables fast and efficient TTS for memory-constrained LLM reasoning. After analyzing common patterns across various TTS methods and identifying their performance bottlenecks, we introduce three novel techniques: *i)* Speculative Beam Extension, which mitigates system stragglers caused by irregular reasoning paths, *ii)* Asymmetric Multi-Model Memory Allocation, which dynamically balances memory usage between token generation and reasoning-step verification, and *iii)* Dynamic Prefix-Aware Scheduling, which optimizes reasoning execution to maximize KV-cache reuse across search paths. *FastTTS* offers a plug-and-play third-party library on top of vLLM, enabling edge LLMs (≤ 7B) on a single consumer GPU (24 GB) to match cloud-model accuracy and cloud-measured latency. Comprehensive evaluation shows that *FastTTS* achieves an average 2.2× higher goodput and reduces latency by 38%–68% compared to the vLLM baseline; it pushes the boundaries of low-latency Test-Time Scaling on memory-constrained edge devices and highlights the potential for democratizing agentic AI.

CCS Concepts: • **Computer systems organization** → **Real-time systems**; • **Computing methodologies** → **Multi-agent planning**.

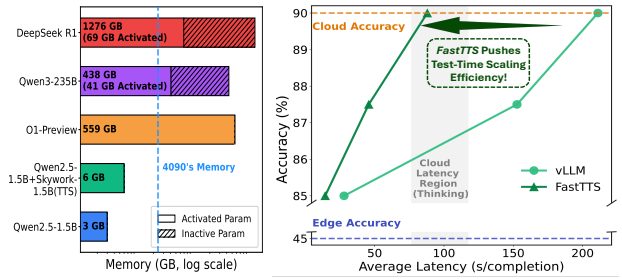
Keywords: machine learning system; LLM reasoning; test-time scaling (TTS); resource-constrained inference

ACM Reference Format:
Hao Mark Chen, Zhiwen Mo, Guanxi Lu, Shuang Liang, Lingxiao Ma, Wayne Luk, and Hongxiang Fan. 2026. FastTTS: Accelerating Test-Time Scaling for Edge LLM Reasoning. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26), March 22–26, 2026, Pittsburgh, PA, USA.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3779212.3790161

1 Introduction

Recent advances in reasoning LLMs have unlocked significant progress in solving complex tasks such as multi-hop question answering, tool use, and long-horizon planning [19, 54, 63]. These capabilities are foundational for agentic AI systems, where AI agents can plan, act, and interact autonomously. As such systems move closer to real-world deployment, there is a growing demand to deploy strong reasoning LLMs at the edge (e.g., on AI PCs), where agentic systems can preserve data privacy, enable personalization, operate offline or with limited connectivity, and interact

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
ASPLOS '26, Pittsburgh, PA, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2359-9/2026/03
https://doi.org/10.1145/3779212.3790161



(a) Memory constraints **(b)** Reduced latency by *FastTTS*

Figure 1. (a) Memory cost across models. (b) *FastTTS* enables low-latency edge deployment of reasoning models. Cloud accuracy: GPT-o1-preview. Edge accuracy: Qwen2.5-Math-1.5B. Cloud latency from the first-answer latency of GPT-o3-pro and GPT-5 (thinking models) [1, 65].

with local environments using high-level intelligence. However, edge hardware imposes severe memory limitations (e.g., a single consumer GPU with 8–24 GB VRAM), restricting deployment to edge LLMs ($\leq$ 7B) that cannot match the reasoning performance of large cloud models, limiting their effectiveness in complex tasks. As shown in Fig. 1a, the memory capacity of consumer-grade GPUs restricts deployment to models like Qwen2.5-Math-1.5B, resulting in a significant gap in reasoning ability compared to large-scale cloud LLMs.

Deploying strong reasoning LLMs on the edge is essential for realizing **democratized agentic AI**, where intelligent agents are decentralized and run directly on client-side devices for better privacy and local integration. Such guarantees are particularly critical in sensitive domains such as healthcare, autonomous driving, and defense, where on-device reasoning is essential for privacy, energy efficiency, or responsiveness [42, 48]. To achieve this, Test-Time Scaling (TTS) [5, 51] has recently emerged as a promising candidate to bridge the reasoning gap between small, edge-deployable LLMs and large cloud-based models. Instead of relying on scaling model parameters during training, TTS allocates additional compute during inference to improve generation and reasoning quality. Despite its great potential, deploying TTS naively on existing serving systems incurs significant latency overhead, making it impractical for real-time applications. As shown in Fig. 1b, using a baseline vLLM implementation to match the accuracy of a large cloud model results in 200 seconds of latency, nearly doubling the latency of large models on cloud infrastructure. To realize the vision of democratized agentic AI, there is an urgent need for an efficient, edge-ready serving infrastructure that makes TTS both performant and practical.

To build such a system, this work begins by first analyzing mainstream TTS methods and abstracting their common execution patterns (Sec. 3.1). We observe that most TTS methods follow a common verifier-guided search pattern that iteratively expands a tree of reasoning paths, where different TTS methods can be viewed as variants or subsets of this approach. Building on this finding, we next conduct a systematic profiling of this common pattern in TTS methods to identify the system-level bottlenecks that hinder its efficiency. Our analysis reveals the following three challenges:

- ***Challenge-1: Hardware Underutilization from Irregular Search Paths.*** Advanced TTS methods employ multi-step, verifier-guided generation, where each search path may produce a variable number of tokens per reasoning step. This divergence leads to execution stragglers, causing idle GPU resources and severely degrading hardware utilization. (Sec. 3.2.1)
- ***Challenge-2: Suboptimal Exploitation of Dynamic Prefix Sharing.*** The parallel search in TTS creates substantial opportunities for prefix-caching reuse, as many generation paths share common thinking prefixes. However, these sharing patterns are dynamic and only known at run-time. Naive scheduling ignores this locality, causing KV cache eviction and re-computation, which is especially severe on memory-constrained edge devices. (Sec. 3.2.2)
- ***Challenge-3: Constrained Memory for Multi-Model Execution.*** A core component of many TTS methods is the use of a separate verifier model to guide the generator. This requires collocating two distinct models in the constrained memory of a consumer-grade GPU. It leads to higher latency due to limited batch size, thereby undermining the benefits of TTS. (Sec. 3.2.3)

To overcome these obstacles, we present ***FastTTS***, a serving system that integrates three synergistic optimizations to make TTS practical on edge devices. To address *Challenge-1*, we introduce **Speculative Beam Extension** that generates speculatively to hide the latency of irregular workloads. To tackle *Challenge-2* and *Challenge-3*, *FastTTS* combines two memory-aware optimizations: **Dynamic Prefix-Aware Scheduling** reorders execution to maximize KV cache reuse from dynamic prefix sharing, and **Asymmetric Multi-Model Memory Allocation** intelligently partitions memory between the generator and verifier to improve throughput. Together, we push the boundaries of edge deployment of TTS (Fig. 1b), making fast and high-quality reasoning feasible on memory-constrained edge devices.

The main contributions of this paper are threefold:

- We systematically analyze the common execution patterns of modern verifier-guided TTS methods and identify their core system-level bottlenecks with a comprehensive performance profiling.
- We design and implement ***FastTTS***, a high-performance serving system for TTS that incorporates three novel and synergistic optimizations: Speculative Beam Extension, Dynamic Prefix-Aware Scheduling, and Asymmetric Multi-Model Memory Allocation.

- We conduct a comprehensive evaluation on representative edge hardware, demonstrating that *FastTTS* achieves an average 2.2× higher goodput and reduces the latency by 38%–68% compared to the vLLM baseline.

2 Background

2.1 LLM Reasoning

Reasoning is a critical capability for Large Language Models (LLMs), enabling multi-step problem solving and complex decision-making. This reasoning capability is initially established through reinforcement learning methods such as Guided Reinforcement Policy Optimization (GRPO), as exemplified by DeepSeek-R1 [54]. Such RL training fosters emergent abilities like long Chain-of-Thought (CoT) reasoning, which in turn expands the applicability of LLMs to domains including mathematical problem solving [49], scientific discovery [19, 41, 64], coding assistant [17, 20], and multi-hop question answering [54, 63].

2.2 Test-Time Scaling (TTS) Methods

While long Chain-of-Thought (CoT) reasoning enhances the capabilities of LLMs, smaller models still lag significantly behind their larger counterparts [38]. To bridge this gap, TTS increases the computational budget for inference by exploring multiple reasoning paths in parallel [5, 38, 51]. Early TTS methods mainly relied on Best-of-N (BoN) sampling, where an Outcome Reward Model (ORM) selects the best solution from a set of fully generated candidates [11, 58]. However, BoN offers limited guidance during generation and yields less diversity in reasoning path structures (Fig. 2). The introduction of Process Reward Models (PRMs), which evaluate intermediate reasoning steps, has enabled advanced verifier-guided search algorithms such as Beam Search and Monte Carlo Tree Search (MCTS) [36, 51, 55]. These methods follow a generation–verification paradigm: a PRM periodically scores partial solutions, expanding high-scoring trajectories and pruning weak ones, thereby concentrating computation on promising paths [7, 23, 62]. As a result, the LLM produces a diverse reasoning *tree* rather than a single chain.

PRMs are primarily categorized as either discriminative or generative [39]. A discriminative PRM functions as a sequence classifier; in a single forward pass, it takes a full reasoning path as input and outputs a score for each intermediate step [21, 36, 56]. In contrast, a generative PRM is an auto-regressive model that must first generate its own textual critique before providing a final score, a significantly more expensive process [70]. Due to their superior balance of model parameters, reasoning quality, and hardware efficiency, discriminative PRMs are the preferred choice for state-of-the-art, verifier-guided TTS systems, particularly for memory-constrained edge deployment [39]. Hence, our system focuses on discriminative PRMs. In contrast, as noted by [38, 51], multi-step lookahead approaches [9, 57], such as

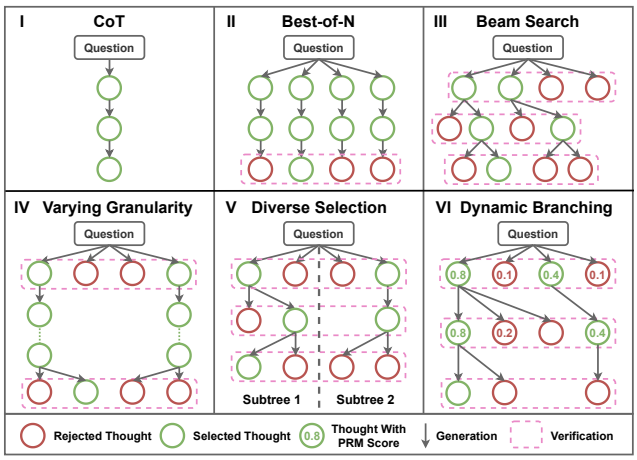


Figure 2. Illustration of different TTS methods.

Monte Carlo Tree Search (MCTS) [12], introduce significant sampling and latency overhead with inferior accuracy, hence we do not consider them in this work.

2.3 LLM Serving

Serving frameworks such as vLLM [30] and SGLang [71] have been developed to optimize throughput and latency in streaming query scenarios. These systems incorporate key optimizations, including KV cache management to avoid recomputing attention states, paged attention to reduce GPU memory fragmentation, and preemptive scheduling to handle memory constraints by swapping requests. For TTS serving, goodput will be a more useful metric rather than throughput, as not all generated tokens will be selected for the final output. Despite its importance, no serving system to date natively supports the structured, multi-path search required for TTS in reasoning tasks.

3 Motivation

In Sec. 3.1, we analyze common patterns in recent TTS methods. Subsequently, we conduct performance profiling and identify the key performance bottlenecks in Sec. 3.2.

3.1 Patterns Analysis in TTS Methods

Recent advancements in LLM reasoning have led to a variety of TTS methods, evolving from simple parallel sampling to more sophisticated search methods [5, 7, 23, 51, 68]. As illustrated in Fig. 2, this evolution marks a structural shift in the generation process: from exploring parallel but independent chains (e.g., CoT and Best-of-N) to constructing structured reasoning trees that allow for intermediate guidance and pruning (e.g., Beam Search, MCTS, and their variants). While recent studies show that small models can attain strong reasoning ability via long sequential CoT [54], tree-structured reasoning remains essential in edge settings.

First, it is orthogonal to sequential CoT and can further enhance the performance of small models. Second, parallel tree-based search has been demonstrated to be substantially more latency-efficient and token-efficient than purely sequential scaling [18, 59], which is critical for maintaining responsiveness under the tight resource constraints of edge deployment.

While these methods vary in their specific heuristics, they share a common underlying execution pattern: a verifier-guided search that iteratively expands a tree of reasoning paths. This process can be generalized into a two-stage loop:

1. **Generation:** From a set of active reasoning paths (*beams*), the generator extends each one by generating a new thinking step, which consists of an arbitrary number of tokens.
2. **Verification:** A PRM, or verifier, evaluates each newly generated step and assigns a score. Top-scoring paths are then replicated to spawn the next set of active beams, while the rest are pruned.

This two-step process repeats until all reasoning paths reach a terminal state. Various search algorithms shown in Fig. 2 can be understood as specific implementations of this general pattern, differing in the heuristics they apply during the Generation or Verification stage. For instance, during Verification, standard **Beam Search** selects the top-K candidates globally with a static branching factor. In contrast, **Diverse Selection** [5, 51] modifies this to improve diversity by choosing the top candidate from distinct subtrees, while **Dynamic Branching** [23, 62] makes the branching factor itself adaptive to verifier scores and system state. Other methods, like **VG-Search** [7], instead modify the Generation stage by altering the length of the thinking steps with varying verification granularities.

To understand the accuracy–latency trade-offs across different TTS methods, we conduct evaluations on the MATH-500 dataset. As illustrated in Fig. 3 (left), while advanced search methods often achieve higher algorithm accuracy, their overall latency remains a critical bottleneck. To address this system-level performance gap, we analyze the challenges shared by the abstracted TTS pattern.

3.2 Challenge Analysis and Performance Profiling

3.2.1 Hardware Underutilization from Irregular Workloads. A core challenge in serving verifier-guided TTS methods stems from the highly irregular and unpredictable workloads they create. Unlike simple token-level generation, the number of tokens generated from a thinking step between verifications can vary dramatically across parallel search paths. We analyze the distribution of these step lengths on the AIME dataset. As shown in Fig. 3 (right), the disparity is extreme. This vast difference between the average and outlier path lengths persists across all steps.

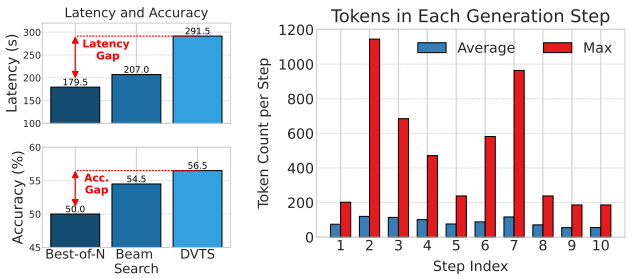


Figure 3. Left: Accuracy vs. latency for different TTS methods on MATH-500 datasets. Right: Avg. and max. token count per generation step of Qwen2.5-Math-1.5B on AIME.

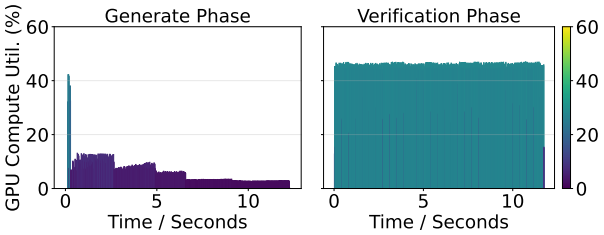


Figure 4. GPU compute utilization in generation and verification phases over time. Irregular during the generation phase. The metrics are collected using Nsight Systems, NVIDIA's official profiling tool, at a sampling rate of 10,000 samples per second for the Tensor Core utilization metrics.

This workload irregularity leads directly to severe hardware underutilization. In a batch of parallel beams, the system must wait for the longest path, known as the "straggler", to complete before proceeding to the next verification stage. As shorter paths finish early, GPU resources are left idle, leading to inefficient resource utilization. Fig. 4 visualizes this problem using GPU compute utilization metrics from Nsight Systems [43]. During the generation phase, utilization peaks at the start but then plummets and progressively decays as more beams complete, leaving the GPU underutilized while waiting for the final straggler. This stands in stark contrast to the consistently high utilization seen during the verification phase (Fig. 4), where workloads are uniformly prefilling. This issue is especially pronounced in edge settings, where small batch sizes render continuous batching inapplicable. Such divergence leaves hardware resources idle and significantly increases end-to-end latency.

3.2.2 Suboptimal Exploitation for Dynamic Prefix Sharing Under Limited Memory. The tree-like exploration of reasoning paths in TTS creates a significant opportunity for memory optimization through KV cache sharing, as shown in Fig. 5 (left). The importance of exploiting such opportunities becomes particularly important under

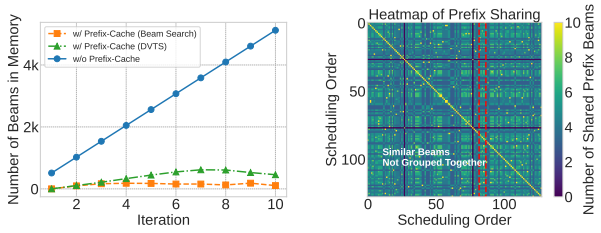


Figure 5. Optimization opportunity in Dynamic Prefix-Cache Sharing. Left: Prefix-cache sharing enables potential substantial memory savings for different TTS methods. Right: Naive scheduling overlooks the dynamic nature of prefix-cache sharing.

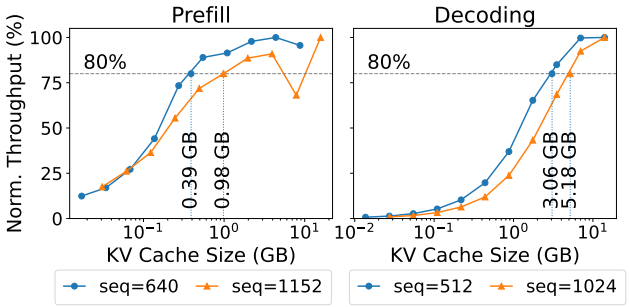


Figure 6. Normalized throughput versus KV cache size in prefill and decoding stages. Prefill saturates much easily.

tight memory constraints for TTS reasoning. Since multiple beams often share a common prefix, scheduling these beams together in a batch enables KV cache reuse and avoids frequent cache eviction. A scheduling policy that exploits this locality can also enable a larger effective batch size within a constrained memory budget. However, these prefix-sharing patterns are dynamic and only emerge at run-time as the reasoning tree expands. The current scheduling policy does not address this, as shown in Fig. 5 (right). This necessitates a dedicated run-time scheduling policy that can maximize KV cache reuse, thereby minimizing redundant computation and memory access.

3.2.3 Constrained Memory for Multi-Model Execution. While TTS is deployable on edge devices with smaller models, its performance is severely hampered by constrained GPU memory on the edge (Fig. 1b). Verifier-guided search on a single device inherently requires collocating multiple models and accommodating potentially large search widths, which together place significant pressure on memory resources. Previous work has shown that LLM throughputs are greatly affected by available GPU memory, which determines the maximum batch size [30, 50]. Addressing this bottleneck is therefore critical for improving LLM reasoning performance on edge devices.

In a memory-constrained TTS system, the generator and verifier share the same limited pool of KV cache memory. However, these two components exhibit vastly different throughput sensitivities to their allocated memory. The verifier, which processes prompts in large batches (prefill), is typically compute-bound, while the generator, which decodes tokens one by one, is memory-bandwidth bound and highly sensitive to KV cache size. This is demonstrated in Fig. 6, which shows that the verifier's prefill stage reaches 80% of its peak throughput with less than 1 GB of KV cache. In contrast, the generator's decoding stage requires 5–10× more memory to reach the same relative throughput. This performance asymmetry reveals a key opportunity: instead of partitioning memory arbitrarily, a carefully profiled, asymmetric allocation can significantly improve overall system throughput by providing each component with the optimal amount of memory.

4 *FastTTS*: Method and Optimization

4.1 Speculative Beam Extension

To mitigate the inefficiency from irregular thinking step lengths (Sec. 3.2.1), we propose **Speculative Beam Extension**, a technique that opportunistically leverages this underutilized hardware. The key idea is to speculatively generate future tokens for beams with short thinking steps in the current iteration, effectively overlapping computations and hiding the latency of stragglers. The high-level procedure is detailed in Fig. 7 and Algorithm 1.

The core logic resides in the generation `while` loop (lines 7–14), which runs until all beams ($\in B$) complete their current generation step. Beams selected for speculative generation are referred to as *speculative candidates*. Within the loop, the system generates one token for both unfinished requests and speculative candidates (line 10), then updates the finished-beam set. From the newly finished beams, *SelectSPEC* (line 12) chooses the most promising candidates, as detailed in Sec. 4.1.1. Once all beams are completed, the algorithm enters the standard verification and selection phase (lines 15–17). We verify beams without considering speculative tokens to ensure **algorithmic equivalence**. Finally, we duplicate all selected beams for branching. If a beam underwent speculation, only its duplicates have speculative tokens truncated (lines 18—19), while the original remains intact to simulate divergence. The truncation length is drawn from a normal distribution with mean R.

4.1.1 Speculative Candidate Selection. To maximize the benefit of speculative execution, our selection of *speculative candidates* is guided by a two-fold objective: minimizing the system overhead incurred during the process, and maximizing the probability that the speculative work will be useful.

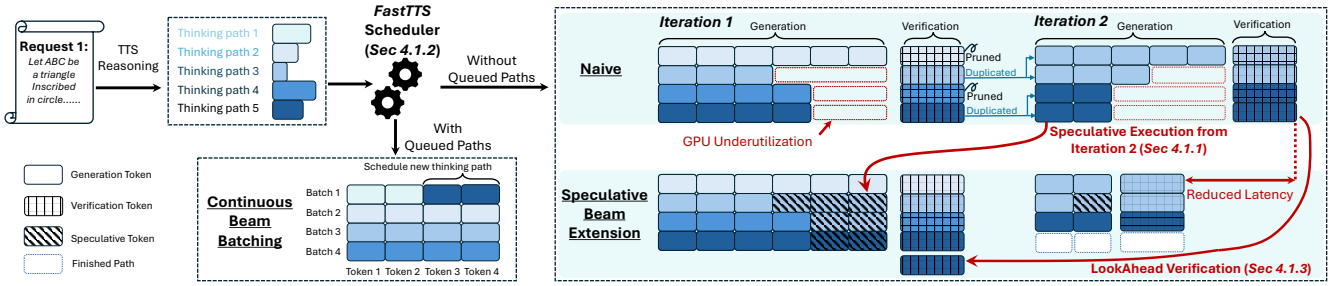


Figure 7. Speculative Beam Extension.

To maximize the utility of speculative execution while maintaining algorithmic equivalence, we use a low-cost heuristic to prioritize how speculative compute resources are allocated. As verifier scores between consecutive steps are often correlated [7], the score from the previous step serves as an effective, zero-overhead proxy for a beam's probability of being retained by the search algorithm. Our system policy partitions these scores into B discrete bins, $\{C_1, \ldots, C_B\}$, where C_1 is the highest-score bin and B is the search's branching factor. For a beam b_i with score s_i, our policy determines its *speculative potential*—the theoretical maximum number of branches it is eligible to generate speculatively, M_i:

$$\text{If } s_i \in C_j, \quad \text{then } M_i = B - j + 1.$$

The value M_i serves as an upper bound and a scheduling priority. In practice, the actual number of speculative branches is determined opportunistically. To maintain a constant batch size and avoid introducing latency, speculative work is performed lazily: as standard beams in the batch complete, the newly available execution slots are filled by speculative branches from the highest-priority completed beams (i.e., those with the highest M_i). The policy thus dynamically allocates a larger compute budget to the beams most likely to be chosen by the unmodified search algorithm, increasing the probability that the speculative work will be useful without altering the final outcome.

4.1.2 Two-Phase Scheduling with Preemption.

To improve GPU utilization without introducing latency overhead or harming responsiveness, we introduce a two-phase, preemptible scheduling policy tailored for TTS workloads. Unlike traditional inference where continuous batching is only effective across multiple user requests, a single TTS request decomposes into many parallel reasoning paths. This unique structure allows for a special form of continuous batching *within* a single request, which we term **Continuous Beam Batching**. Our scheduler leverages this opportunity in a two-phase approach:

- **Phase 1: Continuous Beam Batching.** The scheduler's primary mode is to continuously batch the parallel thinking paths generated by the active TTS request from the waiting queue. This reduces the latency of a single request by maximizing GPU throughput of all thinking paths.
- **Phase 2: Speculative Execution.** When all available reasoning paths are being processed with an empty waiting queue, it transitions to the speculative phase. In this phase, it performs **Speculative Beam Extension** on completed beams to keep the execution batch full, effectively hiding straggler latency.

The speculative phase is fully preemptible: if a new request arrives or a running request is preempted due to memory constraints, all speculative generation is immediately stopped, and the system reverts to Phase 1 to serve the new request. This two-phase design ensures minimal overhead and preserves low-latency responsiveness. Crucially, the speculative phase does not introduce extra tail latency, as all speculative executions are strictly terminated—regardless of their progress—once all the standard beam generations for the current step finish.

Algorithm 1 *Speculative Beam Extension*

```
function SpecBeamExtend(B, R)
    Input: Set of active beams B, Truncation Ratio R
    Output: Next set of beams B_next
    ▷ Generation with Speculation
    B_finished ← ∅
    B_spec ← ∅
    while B_stragglers ≠ ∅ do
        B_stragglers ← B \ (B_finished ∪ B_spec)
        B_running ← B_stragglers ∪ B_spec
        B_new_finished ← GenerateOneToken(B_running)
        B_finished ← B_finished ∪ B_new_finished
        B_new_spec ← SelectSpec(B_new_finished \ B_spec)
        B_spec ← B_spec ∪ B_new_spec
    end while
    ▷ Verification and Selection
    Scores ← Verifier.Evaluate(B)
    B_selected ← Select(B, Scores)
    ▷ Branching and Truncation
    B_selected ← DuplicateThenTruncate(B_selected, R)
    return B_selected
end function
```

4.1.3 LookAhead Verification.

A key optimization opportunity arises when Speculative Beam Extension produces an entire future CoT step for a candidate beam. In a standard pipeline, this would trigger two separate verifier calls across iterations, one for the current step and another for the speculative step in the next. We address this with **LookAhead Verification**, which exploits the verification locality created by speculation. Instead of verifying the two steps separately, we concatenate the output of the current step with the speculative step and submit them together as a single verifier request in the current iteration. If the speculative path is ultimately chosen, this reduces total verifier latency by improving **KV cache locality**. Processing the two adjacent steps as a continuous sequence allows the verifier to reuse the same KV cache, avoiding costly evictions due to limited memory and eliminating the potential need to recompute key–value states in the next iteration.

4.2 Dynamic Prefix-Aware Scheduling

Based on the motivation to exploit the unique temporal locality in the generation phase of verifier-guided TTS, our objective is to minimize KV cache evictions over time by intelligently ordering computations using Dynamic Prefix-Aware Scheduling (Fig. 8). We first frame this as an optimization problem. At each iteration of the generation process, the scheduler receives a list of active reasoning paths, or CoTs, where each CoT is a sequence of beams. A schedule, S, determines the processing order for this list of CoTs. Given a constrained KV cache memory budget, the ordered list is partitioned into batches. Each batch is represented as a radix tree (Trie), T_i, which is the largest possible group of consecutively scheduled CoTs that can fit into memory. Within a Trie, each node represents a unique beam.

We model the cost of KV cache eviction when switching from processing Trie T_i to T_{i+1} as the number of old nodes that must be evicted from memory. The total eviction cost is the sum of these costs over the entire schedule:

$$\text{Cost} = \sum_i (\text{Nodes}(T_i) - P(T_i, T_{i+1}))$$

Here, $\text{Nodes}(T_i)$ is the node count of Trie T_i, and $P(T_i, T_{i+1})$ is the size of the shared prefix (i.e., the number of common nodes) between the two consecutive Tries. To facilitate our analysis, we assume that the sum $\sum_i \text{Nodes}(T_i)$ is constant. A complete list of assumptions is provided in Appendix A.1. Minimizing the eviction cost is equivalent to maximizing the sum of shared prefixes. Therefore, the optimization problem is to find the schedule S^* that achieves this:

$$S^* = \underset{S}{\text{argmax}} \left(\sum_i P(T_i, T_{i+1}) \right)$$

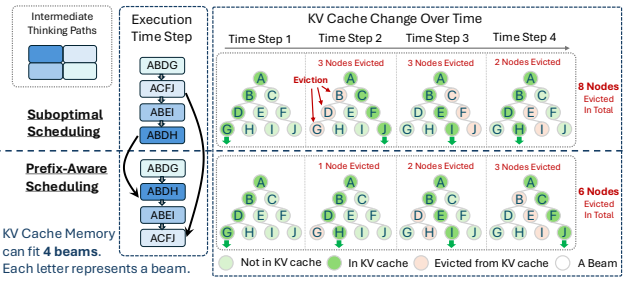


Figure 8. Dynamic Prefix-Aware Scheduling. Example showing how reordering intermediate thinking paths reduces KV cache eviction. Input thinking paths are stored in memory as a Radix Tree. For clarity, the KV cache of newly generated output tokens is omitted in the diagram.

We solve this optimization problem using a greedy approach. Given the set Q of CoTs to be scheduled, the following scheduling invariant is maintained:

$$T_{k+1} = \underset{c_i \in Q}{\text{argmax}}\, P(c_k, c_i)$$

In practice, the local maximization strategy serves as an effective heuristic, often approaching the global optimum empirically. We establish the local optimality of this strategy under certain assumptions. A formal proof, based on a pairwise interchange argument, is provided in Appendix A.2. We implement this greedy approach efficiently by grouping beams spawned from the same parent beam within the scheduling queue, while preserving the relative order of the parent beams across iterations.

4.3 Asymmetric Multi-Model Memory Allocation

4.3.1 Roofline-Guided KV Allocation.

As established in our motivation (Sec. 3.2.3), the available KV cache memory greatly affects system throughput. Statically partitioning memory between the verifier and generator is often suboptimal due to their distinct compute patterns. We therefore propose a roofline-guided KV allocation strategy that balances the KV cache between the generator and verifier to maximize overall system throughput (Fig. 9).

Formulation. Our goal is to find the optimal batch sizes for the prefill (verifier) stage, B_{pre}, and the decoding (generator) stage, B_{dec}, that minimize the total execution time, T_{tot}, for a workload of N requests. We define the total time T_{tot} as the sum of the time spent in each stage:

$$T_{\text{tot}} = \underbrace{\left\lceil \frac{N}{B_{\text{pre}}} \right\rceil T_{\text{roof}}^{\text{pre}}(B_{\text{pre}}, S)}_{\text{Total Prefill/Verifier Time}} + \underbrace{\left\lceil \frac{N}{B_{\text{dec}}} \right\rceil S_{\text{dec}}\, T_{\text{roof}}^{\text{dec}}(B_{\text{dec}}, \bar{S}_{\text{cache}})}_{\text{Total Decoding/Generator Time}},$$

where S is the input sequence length for the verifier, S_{dec} is the generation length for the generator, and $\bar{S}$cache is the average KV cache length during decoding ($\approx S_{\text{dec}}/2$). The term $\lceil \frac{N}{B} \rceil$ calculates the number of batches required to

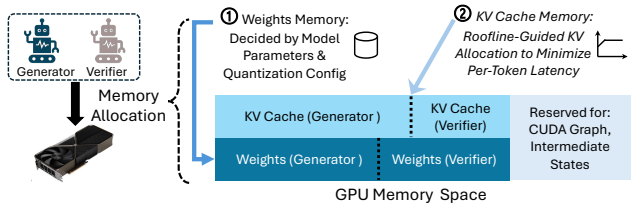


Figure 9. Asymmetric Multi-Model Memory Allocation.

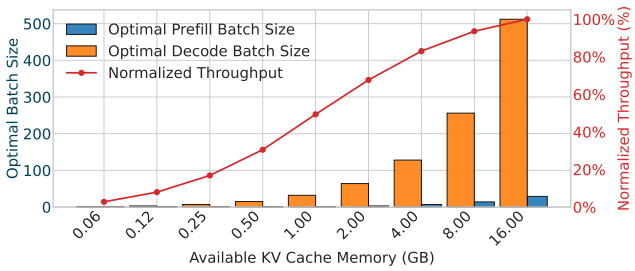


Figure 10. Roofline-Guided KV Allocation. Bars: optimal prefill/decoding batch sizes; line: normalized throughput (higher is better).

process all N requests. For the decoding stage, the per-token generation time is multiplied by the decoding horizon S_{dec}.

This optimization is subject to the total KV cache memory budget, M:

$$B_{\text{pre}} \cdot \text{KVBytes}(1, S) \;+\; B_{\text{dec}} \cdot \text{KVBytes}(1, S_{\text{dec}}) \;\leq\; M.$$

The latency for a single batch in each stage, T_{roof}, is estimated using a standard Roofline model. This model defines latency as the maximum of the time constrained by compute or by memory bandwidth:

$$T_{\text{roof}} = \max\left(\frac{\text{FLOPs}}{P}, \frac{\text{Bytes}}{\text{BW}}\right),$$

where P is the device's peak compute and BW its peak memory bandwidth, per hardware specification.

Search Algorithm. Since the objective function T_{tot} is not necessarily convex, we employ a simple and fast linear search that is guaranteed to find the global optimum. A key insight is that since stage latency monotonically decreases with more memory, the optimal allocation will always lie on the *boundary* of the memory constraint, fully utilizing the available budget M.

Our search algorithm therefore iterates through all feasible integer values for the prefill batch size, B_{pre}. For each candidate B_{pre}, we calculate the maximum possible decoding batch size, B_{dec}, that satisfies the memory constraint:

$$B_{\text{dec}} = \left\lfloor \frac{M - B_{\text{pre}}\text{KVBytes}(1,S)}{\text{KVBytes}(1,S_{\text{dec}})} \right\rfloor. \tag{1}$$

We then evaluate T_{tot} for this $(B_{\text{pre}}, B_{\text{dec}})$ pair and record the pair that yields the minimum total time. Because the decoding stage is typically more sensitive to memory, any ties are resolved in favor of a larger B_{dec}. This entire search process is computationally trivial, averaging < 1 **ms** on a single CPU thread, and thus introduces negligible overhead. Fig. 10 shows an example resulting policy. At run-time, the Roofline-Guided KV Allocation policy is dynamically invoked upon system state changes to quickly adapt the verifier and generator batch sizes.

4.3.2 Extended Search Space with Offloading.

The optimization space can be extended with an offloading strategy for cases where GPU memory M is extremely constrained. Here, the KV cache of the inactive model is offloaded to CPU memory, enabling a single model to fully utilize the GPU cache space and relaxing the coupled constraint into two independent ones:

$$B_{\text{pre}} \cdot \text{KVBytes}(1, S) \leq M, \quad B_{\text{dec}} \cdot \text{KVBytes}(1, S_{\text{dec}}) \leq M.$$

This incurs a transfer overhead $T^{\text{offload}}_{\text{overhead}}$. The system then selects the lower-latency strategy: *i)* the optimal execution time T_{tot} from allocation search under the original constraint, or *ii)* the offloading time $T^{\text{offload}}_{\text{tot}} + T^{\text{offload}}_{\text{overhead}}$, where $T^{\text{offload}}_{\text{tot}}$ is computed from the maximum batch sizes allowed by the relaxed constraints. This dual-strategy policy lets *FastTTS* always pick the better option.

5 Implementation

FastTTS is implemented in ~6,500 lines of Python on top of vLLM (v0.9.2), operating the generator and verifier in separate worker processes via Python's multiprocessing library. We extend the core `LLMEngine` of vLLM to implement our two-phase, preemptive scheduling policy, which dynamically switches between continuous batching and **Speculative Beam Extension** based on the request queue status. For the generator, we extend the default scheduler with our **Dynamic Prefix-Aware Scheduling** that implements a greedy heuristic to group beams from the same parent, maximizing KV cache reuse. Our **Asymmetric Multi-Model Memory Allocation** policy is managed by a lightweight searcher that is invoked dynamically to determine the partition of the KV cache between workers. The system exposes a configurable interface for various TTS strategies and hyperparameters.

6 Evaluation

6.1 Experimental Setup

Platform. All experiments are conducted on a single NVIDIA GeForce RTX 4090 GPU (24 GB VRAM), representing a typical edge device. It is equipped with an Intel Xeon Silver 4310 CPU @ 2.10 GHz. The software stack includes CUDA Toolkit 12.4 with its corresponding versions of Nsight Systems and Nsight Compute, PyTorch 2.7.0, and Python 3.11.

Models. To assess *FastTTS* under diverse workloads, we evaluate three generator-verifier configurations designed to stress different system aspects, following common practice

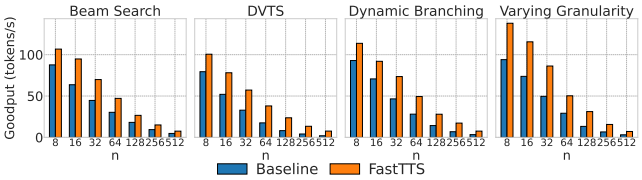


Figure 11. Precise Goodput improvement of *FastTTS* over the vLLM baseline across different search algorithm variants. Experiments use the 1.5B+1.5B configuration on AIME. In dynamic branching, each beam branches proportionally to its verifier score; in varying granularity, the maximum step length is 64 tokens for the first 3 steps and 2048 thereafter.

in prior work [5, 7, 38, 51]. We test a verifier-heavy setting (**1.5B+7B**: Qwen2.5-Math-1.5B generator with a Math-Shepherd-Mistral-7B verifier) and a generator-heavy setting (**7B+1.5B**: Qwen2.5-Math-7B generator with a Skywork-o1-Open-PRM-1.5B verifier), both allocate 90% of GPU memory to test throughput limits. To simulate a highly resource-limited environment, we also test a memory-constrained setting (**1.5B+1.5B**: a 1.5B generator and verifier) [21, 56, 64], restricting it to 40% of GPU memory.

Datasets. We evaluate on two common mathematical reasoning benchmarks [49, 64] of varying difficulty to assess performance under diverse and complex workloads:

- **AIME2024** [40]: A challenging dataset from the American Invitational Mathematics Examination.
- **AMC2023** [4]: A dataset from the American Mathematics Competitions, which presents a broader range of difficulty.

For experiments, we use the test sets of these benchmarks with a batch size of 1 to reflect interactive edge scenarios.

Baseline Implementation. Our baseline system is built on top of the widely-used **vLLM** framework (version 0.9.2). We implement a standard verifier-guided test-time search, running the generator and verifier as separate vLLM instances, with remaining details following Hugging Face's official *search-and-learn* implementation [5]. This baseline represents a naive but robust implementation of TTS, against which we compare the performance gains achieved by the optimizations in *FastTTS*.

Metrics. To provide a comprehensive evaluation of system performance for TTS, we use the following metrics:

- **Precise Goodput**: Standard goodput metrics are often insufficient for TTS tasks. To fairly evaluate system efficiency, we propose a metric termed **Precise Goodput**[1], defined as:

$$\text{Precise Goodput} := \frac{\text{Average token length per beam}}{\text{Average beam completion time}}$$

  This metric is designed to be robust against several sources of evaluation unfairness. Using the average completion time and token length across all beams prevents the metric from being affected by a single slow reasoning path or being artificially inflated by a large number of finally collected paths. Furthermore, it provides a true measure of generation efficiency, unaffected by the copying of text during branching.
- **Completion Time**: We measure the average end-to-end time taken per completion for a problem.

[1]We used Precise Goodput and Goodput interchangeably in this paper.

6.2 End-to-End Performance Improvement

We first evaluate the end-to-end performance of *FastTTS* against the vLLM baseline across a diverse set of popular test-time search algorithms. As shown in Fig. 11, *FastTTS* consistently and significantly improves precise goodput over the baseline implementation across all evaluated search methods. The goodput improvement ranges from 1.2× to 3.9×. DVTS, Dynamic Branching, and Varying Granularity are fundamentally variants of the core beam search algorithm. As beam search represents the most common and foundational use case, we focus the remainder of our evaluation on this representative search method.

Precise Goodput. Fig. 12 shows that *FastTTS* consistently and significantly improves system goodput over the vLLM baseline across all tested scenarios. For all three model configurations (1.5B+1.5B, 1.5B+7B, and 7B+1.5B) and number of beams (n) values from 8 to 512, *FastTTS* achieves an average goodput improvement of **2.2×**, ranging from **1.2× to 5.4×**. These substantial gains stem from our synergistic optimizations: Speculative Beam Extension enhances GPU utilization, while Asymmetric Multi-Model Memory Allocation and Dynamic Prefix-Aware Scheduling improve the efficiency of KV cache management. The relative goodput improvement becomes more pronounced at larger values of n, peaking at 5.4× for the 7B+1.5B configuration at n=512 on AIME. This trend holds for all model pairs, as a larger search budget (n) creates more diverse reasoning paths, further exacerbating hardware underutilization and KV cache pressure—the very issues our optimizations address.

Completion Latency. Beyond improving goodput, *FastTTS* also delivers substantial reductions in end-to-end completion latency. Fig. 13 shows that *FastTTS* achieves an average latency reduction of **38% to 68%** across all configurations and n values compared to the vLLM baseline.

The latency breakdown within *FastTTS* reveals the distinct performance characteristics of each model configuration. In the 7B+1.5B configuration, generator latency (unfilled portion) is the dominant cost. Conversely, in the 1.5B+7B configuration, the larger 7B verifier model contributes a substantial portion of the total latency, becoming nearly on par with the generator as n increases.

FastTTS effectively reduces both the generation and verification components of latency. On average, it reduces verifier latency by **75% to 85%** and reduces generation latency by **36% to 66%** across all n values. The dramatic reduction in

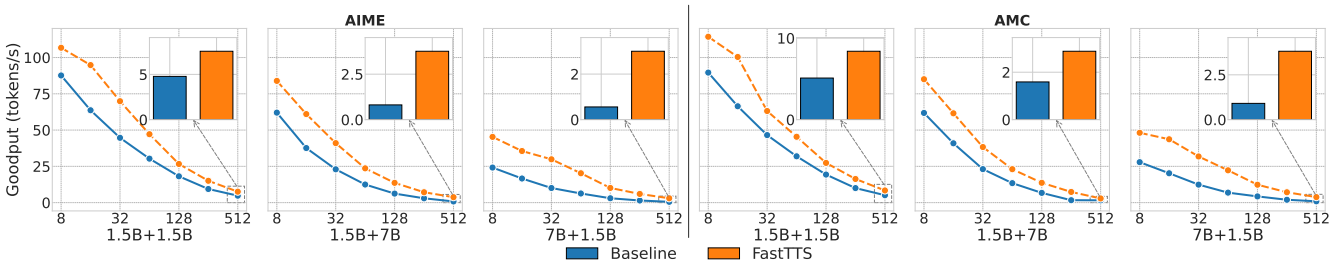


Figure 12. *FastTTS* Goodput Improvement. The x-axis represents the number of beams (n).

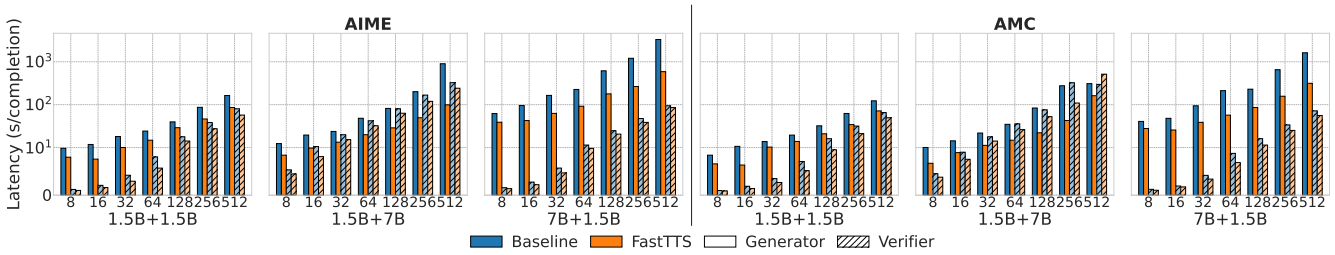
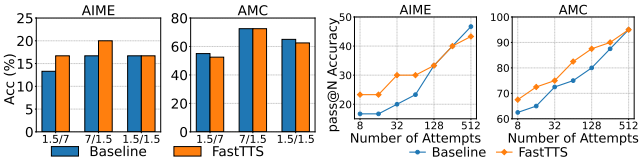


Figure 13. Completion Latency Improvement. The average end-to-end latency for the completion of a single request. The search process is terminated when all active beams reach the end. The x-axis represents the number of beams.



(a) Top-1 Accuracy with n=512. **(b)** Pass@N accuracy.

Figure 14. Algorithm accuracy (e.g., 1.5/7 for 1.5B+7B).

verifier latency is primarily driven by our *LookAhead Verification* technique, which enhances computational locality by pre-verifying tokens. The substantial decrease in generator latency is achieved through the combined effects of our other optimizations: Asymmetric Multi-Model Memory Allocation and Dynamic Prefix-Aware Scheduling enhance KV cache efficiency, while Speculative Beam Extension hides straggler latency by utilizing idle GPU cycles. We note one exception where verifier latency slightly increases (at n=512 for the 1.5B+7B model on AMC), a direct trade-off from our memory allocator prioritizing the heavily-loaded generator by reducing the verifier's KV cache capacity.

6.3 Algorithm Performance

We evaluate the impact of our system optimizations on the quality of the generated solutions from two perspectives. For a practical assessment, we report **Top-1 accuracy**, where the final answer is selected from the generated candidates using majority voting. To better understand the quality distribution of all generated solutions and the capability of the search algorithm, we also report **Pass@N accuracy**. This metric measures the success rate where at least one correct answer is found within a set of N generated solutions. For ranking, the N candidates are selected based on their verifier score.

While *FastTTS* is designed to guarantee algorithmic equivalence with the baseline, minor variations in output can occur since our scheduling optimizations may alter the sampling order. We now analyze these effects.

Top-1 Accuracy. As shown in Fig. 14a, the Top-1 accuracy of *FastTTS* is highly competitive with the baseline. On the more challenging AIME dataset, *FastTTS* consistently matches or slightly improves accuracy, likely because its speculative execution focuses computation on the more promising reasoning paths. In general, both perform comparably, confirming the algorithm equivalence.

Pass@N Accuracy. Fig. 14b shows the Pass@N accuracy, providing insight into the search behavior. In practice, it matches baseline accuracy at large $\boldsymbol{N}$ but slightly exceeds it at small $\boldsymbol{N}$, likely due to a side scheduler effect: speculative extension can let long straggler beams generate beyond their original CoT length, occasionally improving accuracy.

Ultimately, for practical deployment, the Top-1 accuracy achieved through majority voting is the more indicative measure of a system's real-world utility.

6.4 Generality on Hardware and Benchmarks

To demonstrate the generality of *FastTTS* across more resource-constrained GPUs and diverse tasks, we further extend our evaluation to additional devices and code generation workloads.

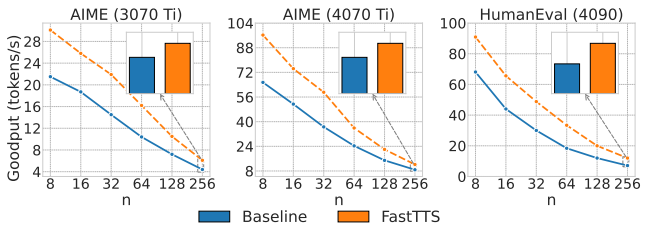


Figure 15. Goodput improvements on constrained hardware and coding tasks. Experiments are conducted on NVIDIA RTX 3070 Ti (8GB) and RTX 4070 Ti (12GB) GPUs using the AIME dataset, and on the HumanEval code generation benchmark using an RTX 4090. Offloading is used on the RTX 3070 Ti; as a result, lower absolute goodput is observed.

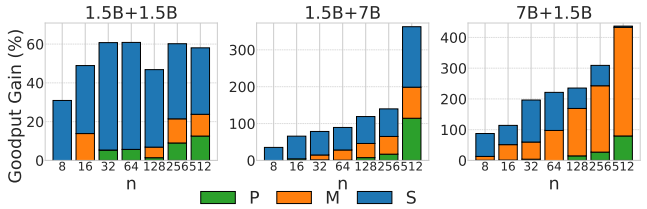


Figure 16. Breakdown of goodput gain from 3 optimizations. The cumulative improvements are shown for Dynamic Prefix-Aware Scheduling (P), Asymmetric Multi-Model Memory Allocation (M), and Speculative Beam Extension (S).

Constrained Hardware. We evaluate *FastTTS* on NVIDIA RTX 3070 Ti (8GB) and RTX 4070 Ti (12GB) GPUs. As shown in Fig. 15, our system consistently outperforms the baseline, achieving goodput speedups of **1.4×–1.6×**. These results indicate that *FastTTS* remains effective on lower-end edge hardware. We note that *FastTTS* is orthogonal to quantization and offloading techniques, which can be incorporated for additional efficiency gains.

Broader Benchmarks. On the HumanEval code generation benchmark, *FastTTS* attains speedups ranging from **1.3× to 1.8×** (Fig. 15). This demonstrates that the execution patterns optimized by *FastTTS* transfer effectively to other complex domains, including code generation.

6.5 Ablation Study

6.5.1 Goodput Gain Breakdown. To understand the individual contribution of each of our proposed optimizations, we conduct an ablation study. The results, shown in Fig. 16, break down the cumulative performance gains for all three model configurations: 1.5B+1.5B, 1.5B+7B, and 7B+1.5B.

Dynamic Prefix-Aware Scheduling (P). This optimization provides a foundational layer of improvement that becomes more apparent as n increases. As shown by the green bars, its gain is most significant in memory-constrained scenarios (e.g., the 1.5B+7B setup), where maximizing prefix reuse is critical. This is intuitive, as a larger number of beams (n) leads to a more constrained KV cache where minimizing evictions is paramount.

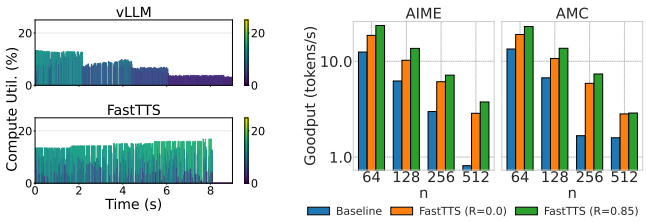


Figure 17. Spec Beam Extension Ablation. Left: Compute Utilization across time in 1 iteration. Right: The impact of the speculative truncation ratio (R) on Goodput.

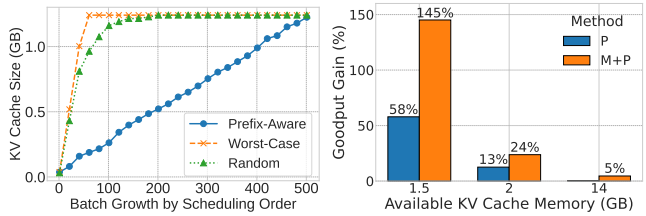


Figure 18. Left: Effectiveness of Dynamic Prefix-Aware Scheduling, using traces from the final TTS iteration (1.5B+1.5B, AIME). The vLLM baseline uses random scheduling. Right: Impact of Memory Availability on Optimization Gains. The chart shows the goodput gain over baselines.

Asymmetric Multi-Model Memory Allocation (M). Adding Asymmetric Multi-Model Memory Allocation on top of Dynamic Prefix-Aware Scheduling delivers additional performance improvement. This component is a major source of improvement across all three model configurations, particularly at larger n. This is because under a high compute budget, intelligently partitioning memory between the generator and verifier is crucial to prevent frequent preemptions and costly re-computation for the generator.

Speculative Beam Extension (S). Speculative Beam Extension consistently provides a significant, and often the largest, performance improvement. This technique offers a substantial goodput gain across almost all scenarios. The improvement is most pronounced when more KV cache memory is available for parallel speculation, such as in the 1.5B+1.5B and 1.5B+7B configurations. By effectively hiding the latency of straggler beams, this optimization improves goodput.

6.5.2 In-depth Study of Speculative Beam Extension. As shown in Fig. 17 (left), Speculative Beam Extension improves hardware utilization. While the baseline vLLM implementation suffers from progressively decaying GPU compute utilization as faster reasoning paths in a batch finish early, *FastTTS* maintains a higher and more consistent utilization by speculatively generating tokens for completed beams.

The overall latency is also reduced, as the speculative tokens generated in one iteration can be used as a head start for the next, shortening their required generation time. The performance of Speculative Beam Extension is also affected by its truncation ratio, *R*. As shown in Fig. 17 (right), a higher ratio (R=0.85), which aggressively retains speculative work, yields more goodput improvement.

6.5.3 Effectiveness of Dynamic Prefix-Aware Scheduling.

We evaluate the memory efficiency of Dynamic Prefix-Aware Scheduling against Random and Worst-Case baselines in Fig. 18 (left). First, KV cache size grows much more slowly with batch size under Dynamic Prefix-Aware Scheduling, indicating higher cache reuse and fewer evictions. The cache size might saturate early due to beam duplication during branching in beam search. Second, given a fixed KV cache budget, Dynamic Prefix-Aware Scheduling supports substantially larger batches, directly improving throughput.

6.5.4 Impact of Memory Constraints on Optimizations.

Fig. 18 (right) illustrates the performance gains from our optimizations under varying memory availability. The effectiveness of both **Dynamic Prefix-Aware Scheduling (P)** and its combination with **Asymmetric Multi-Model Memory Allocation (M+P)** is most pronounced in memory-constrained scenarios. At 1.5 GB of available KV cache, the optimizations deliver substantial goodput gains of 58% and 145%, respectively. However, when memory is relatively abundant (e.g., 14 GB), the benefits diminish. This is because a large memory budget can accommodate the entire batch of reasoning paths, which minimizes the KV cache eviction that Dynamic Prefix-Aware Scheduling is designed to prevent. Similarly, when memory is not a bottleneck, a sophisticated allocation strategy becomes less critical.

7 Related Work

7.1 Reasoning Systems and Speculative Execution

Edge and Memory-Constrained Serving: The recent development of edge LLM serving systems focuses mainly on optimizing the deployment of **non-reasoning workloads** [52, 60, 66, 72].

Efficient Reasoning: Although Certaindex [14] covers LLM reasoning serving, it focuses solely on CoT reasoning with query-level scheduling and early termination. It does not handle the irregular computation patterns within TTS reasoning trajectories, nor does it optimize scheduling between the generator and verifier. Beyond system-level serving optimizations, recent algorithmic advances such as speculative reasoning [15, 35, 44] aim to accelerate inference by leveraging an efficient draft model. However, these approaches modify the output distribution and therefore lack algorithmic equivalence. We emphasize that such algorithmic techniques are orthogonal to our system-level optimizations and can be seamlessly integrated into our framework to achieve further speedups.

Speculative Execution: The philosophy of speculative generation for LLMs was initially explored at the algorithmic level, primarily through speculative and parallel decoding techniques [6, 8, 10, 13, 24, 33, 34, 47]. However, these methods serve as algorithmic enhancements aimed at accelerating LLM decoding by enabling multi-token generation. Speculative generation has also been applied in retrieval-augmented generation (RAG) systems [25, 27, 69], where it is used to prefetch or cache retrieved documents to reduce latency. In contrast, our Speculative Beam Extension targets the unique system-level bottleneck of parallel reasoning: hardware underutilization caused by irregular step lengths. Instead of verifying draft tokens, we utilize idle compute slots from completed paths to speculatively extend future reasoning steps, thereby mitigating straggler effects and maintaining high GPU occupancy without altering the algorithm.

7.2 Memory Management and Prefix Sharing

Beyond the various memory and scheduling optimizations [16, 29, 30, 32, 37, 53, 61, 74], two other methods have emerged for improving memory efficiency in LLM serving: offloading and prefix sharing.

Offloading for LLMs: Offloading is a primary strategy for alleviating LLM memory pressure, typically via computation offloading and data offloading. In computation offloading, several approaches, such as FlexGen [50], FastDecode [22], PowerInfer [52], and NEO [26], distribute LLM pipelines across CPU and GPU to reduce GPU load. LIA [28] explores offloading to Intel AMX, while [67] supports edge–cloud partitioning for inference. In data offloading, DeepSpeed-Inference [3] utilizes CPU host memory for activation offloading, while [2] explores flash-based offloading strategies. AIF [31] pushes this further by supporting in-flash processing to reduce data movement overhead.

Prefix Caching and Sharing: Most prior work on prefix caching and sharing focuses on query-level optimization using tree-structured management [30, 71]. BatchLLM [73] explores the global prefix reuse through ahead-of-time prefix identification. FastTree [45] improves tree-structured inference via context-query grouping to enhance cache locality. RAGCache [27] investigates prefix sharing with dynamic overlapping between the retrieval and inference steps for retrieval-augmented generation. KVFlow [46] further advances prefix scheduling in multi-agent systems by introducing workflow-aware eviction policies and overlapped KV prefetching. However, previous work on prefix optimization has primarily focused on **coarse-grained, query-level sharing** in non-reasoning LLM serving scenarios. In contrast, LLM reasoning workloads introduce new opportunities for fine-grained, prefix-aware sharing during decoding.

8 Conclusion

This paper presents *FastTTS*, a plug-and-play third-party serving system that makes Test-Time Scaling both practical and efficient on memory-constrained edge devices. By analyzing the common execution pattern of mainstream TTS methods, we identify several key system challenges: *i)* hardware underutilization of irregular reasoning search paths, *ii)* suboptimal cache reuse, and *iii)* memory pressure from multi-model execution. *FastTTS* addresses these challenges through three novel synergistic techniques: Speculative Beam Extension, Dynamic Prefix-Aware Scheduling, and Asymmetric Multi-Model Memory Allocation. Our evaluation shows that *FastTTS* enables low-latency, high-quality reasoning using edge LLMs for memory-constrained devices. It narrows their performance gap with cloud-scale models, and advances the vision of democratized agentic AI.

9 Acknowledgement

The support of UKRI and EPSRC (grant numbers UKRI256, EP/V028251/1, EP/N031768/1, EP/S030069/1, and EP/X036006/1), KIAT, AMD and Intel is gratefully acknowledged.

References

[1] 2025. Artificial Analysis: LLM Leaderboard. https://artificialanalysis.ai/leaderboards/models?size_class=large&reasoning=reasoning. Accessed: 2025-08-16.

[2] Keivan Alizadeh, Seyed Iman Mirzadeh, Dmitry Belenko, S Khatamifard, Minsik Cho, Carlo C Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. 2024. Llm in a flash: Efficient large language model inference with limited memory. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 12562–12584.

[3] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, et al. 2022. Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–15.

[4] Mislav Balunović, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. 2025. MathArena: Evaluating LLMs on Uncontaminated Math Competitions. https://matharena.ai/

[5] Edward Beeching, Lewis Tunstall, and Sasha Rush. [n. d.]. Scaling test-time compute with open models. https://huggingface.co/spaces/HuggingFaceH4/blogpost-scaling-test-time-compute

[6] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. In *International Conference on Machine Learning (ICML)*.

[7] Hao Mark Chen, Guanxi Lu, Yasuyuki Okoshi, Zhiwen Mo, Masato Motomura, and Hongxiang Fan. 2025. Rethinking Optimal Verification Granularity for Compute-Efficient Test-Time Scaling. *Advances in Neural Information Processing Systems* (2025).

[8] Hao Mark Chen, Wayne Luk, Ka Fai Cedric Yiu, Rui Li, Konstantin Mishchenko, Stylianos I Venieris, and Hongxiang Fan. 2024. Hardware-aware parallel prompt decoding for memory-efficient acceleration of llm inference. *arXiv preprint arXiv:2405.18628* (2024).

[9] Sijia Chen and Baochun Li. 2024. Toward adaptive reasoning in large language models with thought rollback. *arXiv preprint arXiv:2412.19707* (2024).

[10] Xinhao Cheng. [n. d.]. *SpecInfer: Accelerating Generative Large Language Model Serving with Speculative Inference and Token Tree Verification.* Ph. D. Dissertation. Carnegie Mellon University.

[11] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168* (2021).

[12] Xidong Feng, Ziyu Wan, Muning Wen, Stephen Marcus McAleer, Ying Wen, Weinan Zhang, and Jun Wang. 2023. Alphazero-like tree-search can guide large language model decoding and training. *arXiv preprint arXiv:2309.17179* (2023).

[13] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. 2024. Break the Sequential Dependency of LLM Inference Using Lookahead Decoding. In *International Conference on Machine Learning (ICML)*.

[14] Yichao Fu, Junda Chen, Siqi Zhu, Zheyu Fu, Zhongdongming Dai, Yonghao Zhuang, Yian Ma, Aurick Qiao, Tajana Rosing, Ion Stoica, et al. 2024. Efficiently Scaling LLM Reasoning with Certaindex. *arXiv preprint arXiv:2412.20993* (2024).

[15] Yichao Fu, Rui Ge, Zelei Shao, Zhijie Deng, and Hao Zhang. 2025. Scaling Speculative Decoding with Lookahead Reasoning. *arXiv preprint arXiv:2506.19830* (2025).

[16] Yaoqi Fu, Yanqi Zhang, et al. 2024. ServerlessLLM: Low-Latency Serverless Inference for Large Language Models. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. https://www.usenix.org/system/files/osdi24-fu.pdf

[17] Jiahao Gai, Hao Chen, Zhican Wang, Hongyu Zhou, Wanru Zhao, Nicholas Lane, and Hongxiang Fan. 2025. Exploring code language models for automated hls-based hardware generation: Benchmark, infrastructure and analysis. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference*. 988–994.

[18] Soumya Suvra Ghosal, Souradip Chakraborty, Avinash Reddy, Yifu Lu, Mengdi Wang, Dinesh Manocha, Furong Huang, Mohammad Ghavamzadeh, and Amrit Singh Bedi. 2025. Does Thinking More always Help? Understanding Test-Time Scaling in Reasoning Models. *arXiv preprint arXiv:2506.04210* (2025).

[19] Google DeepMind. 2025. AlphaEvolve: A Gemini-Powered Coding Agent for Designing Advanced Algorithms. https://deepmind.google/discover/blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/. Accessed: 2025-07-08.

[20] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming–The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).

[21] Jujie He, Tianwen Wei, Rui Yan, Jiacai Liu, Chaojie Wang, Yimeng Gan, Shiwen Tu, Chris Yuhao Liu, Liang Zeng, Xiaokun Wang, Boyang Wang, Yongcong Li, Fuxiang Zhang, Jiacheng Xu, Bo An, Yang Liu, and Yahui Zhou. 2024. Skywork-o1 Open Series. https://huggingface.co/Skywork. https://huggingface.co/Skywork

[22] Jiaao He and Jidong Zhai. 2024. Fastdecode: High-throughput gpu-efficient llm serving using heterogeneous pipelines. *arXiv preprint arXiv:2403.11421* (2024).

[23] Coleman Hooper, Sehoon Kim, Suhong Moon, Kerem Dilmen, Monishwaran Maheswaran, Nicholas Lee, Michael W Mahoney, Sophia Shao, Kurt Keutzer, and Amir Gholami. 2025. Ets: Efficient tree search for inference-time scaling. *arXiv preprint arXiv:2502.13575* (2025).

[24] Yunhai Hu, Zining Liu, Zhenyuan Dong, Tianfan Peng, Bradley McDanel, and Sai Qian Zhang. 2025. Speculative decoding and beyond: An in-depth survey of techniques. *arXiv preprint arXiv:2502.19732* (2025).

[25] Zhengding Hu, Vibha Murthy, Zaifeng Pan, Wanlu Li, Xiaoyi Fang, Yufei Ding, and Yuke Wang. 2025. HedraRAG: Coordinating LLM Generation and Database Retrieval in Heterogeneous RAG Serving. *arXiv preprint arXiv:2507.09138* (2025).

[26] Xuanlin Jiang, Yang Zhou, Shiyi Cao, Ion Stoica, and Minlan Yu. 2024. Neo: Saving gpu memory crisis with cpu offloading for online llm inference. *arXiv preprint arXiv:2411.01142* (2024).

[27] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Xin Liu, Xuanzhe Liu, and Xin Jin. 2024. Ragcache: Efficient knowledge caching for retrieval-augmented generation. *arXiv preprint arXiv:2404.12457* (2024).

[28] Hyungyo Kim, Nachuan Wang, Qirong Xia, Jinghan Huang, Amir Yazdanbakhsh, and Nam Sung Kim. 2025. LIA: A Single-GPU LLM Inference Acceleration with Cooperative AMX-Enabled CPU-GPU Computation and CXL Offloading. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 544–558.

[29] Minseok Kim, Jongse Park, et al. 2025. Oaken: Fast and Efficient LLM Serving with Online-Offline Memory Management. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. https://jongse-park.github.io/files/paper/2025-isca-oaken.pdf

[30] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.

[31] Jaeyong Lee, Hyeunjoo Kim, Sanghun Oh, Myoungjun Chun, Myungsuk Kim, and Jihong Kim. 2025. AiF: Accelerating On-Device LLM Inference Using In-Flash Processing. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 529–543.

[32] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. https://www.usenix.org/system/files/osdi24-lee.pdf

[33] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. Eagle-2: Faster inference of language models with dynamic draft trees. *arXiv preprint arXiv:2406.16858* (2024).

[34] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2025. Eagle-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840* (2025).

[35] Baohao Liao, Yuhui Xu, Hanze Dong, Junnan Li, Christof Monz, Silvio Savarese, Doyen Sahoo, and Caiming Xiong. 2025. Reward-guided speculative decoding for efficient llm reasoning. *arXiv preprint arXiv:2501.19324* (2025).

[36] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's verify step by step. In *The Twelfth International Conference on Learning Representations*.

[37] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, and Fan Yang. 2024. Parrot: Efficient Serving of LLM-based Applications with Semantic Variable. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. https://www.usenix.org/system/files/osdi24-lin-chaofan.pdf

[38] Runze Liu, Junqi Gao, Jian Zhao, Kaiyan Zhang, Xiu Li, Biqing Qi, Wanli Ouyang, and Bowen Zhou. 2025. Can 1B LLM Surpass 405B LLM? Rethinking Compute-Optimal Test-Time Scaling. *arXiv preprint arXiv:2502.06703* (2025).

[39] Saumya Malik, Valentina Pyatkin, Sander Land, Jacob Morrison, Noah A. Smith, Hannaneh Hajishirzi, and Nathan Lambert. 2025. RewardBench 2: Advancing Reward Model Evaluation.

[40] Mathematical Association of America. 2024. American Invitational Mathematics Examination (AIME). https://maa.org/math-competitions/american-invitational-mathematics-examination-aime. Accessed February 2024.

[41] Adrian Mirza, Nawaf Alampara, Sreekanth Kunchapu, Martiño Ríos-García, Benedict Emoekabu, Aswanth Krishnan, Tanya Gupta, Mara Schilling-Wilhelmi, Macjonathan Okereke, Anagha Aneesh, et al. 2025. A framework for evaluating the chemical knowledge and reasoning abilities of large language models against the expertise of chemists. *Nature Chemistry* (2025), 1–8.

[42] Leon Nissen, Philipp Zagar, Vishnu Ravi, Aydin Zahedivash, Lara Marie Reimer, Stephan Jonas, Oliver Aalami, and Paul Schmiedmayer. 2025. Medicine on the edge: Comparative performance analysis of on-device LLMs for clinical reasoning. *arXiv preprint arXiv:2502.08954* (2025).

[43] NVIDIA Corporation. 2025. NVIDIA Nsight Systems. https://developer.nvidia.com/nsight-systems Accessed: 2025-04-15.

[44] Rui Pan, Yinwei Dai, Zhihao Zhang, Gabriele Oliaro, Zhihao Jia, and Ravi Netravali. 2025. Specreason: Fast and accurate inference-time compute via speculative reasoning. *arXiv preprint arXiv:2504.07891* (2025).

[45] Zaifeng Pan, Yitong Ding, Yue Guan, Zheng Wang, Zhongkai Yu, Xulong Tang, Yida Wang, and Yufei Ding. 2025. FastTree: Optimizing Attention Kernel and Runtime for Tree-Structured LLM Inference. *Eighth Conference on Machine Learning and Systems*.

[46] Zaifeng Pan, Ajjkumar Patel, Zhengding Hu, Yipeng Shen, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. 2025. KVFlow: Efficient Prefix Caching for Accelerating LLM-Based Multi-Agent Workflows. *arXiv preprint arXiv:2507.07400* (2025).

[47] Zongyue Qin, Zifan He, Neha Prakriya, Jason Cong, and Yizhou Sun. 2025. Dynamic-width speculative beam decoding for llm inference. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 25056–25064.

[48] Jon Saad-Falcon, Avanika Narayan, Hakki Orhun Akengin, J Griffin, Herumb Shandilya, Adrian Gamarra Lafuente, Medhya Goel, Rebecca Joseph, Shlok Natarajan, Etash Kumar Guha, et al. 2025. Intelligence per watt: Measuring intelligence efficiency of local ai. *arXiv preprint arXiv:2511.07885* (2025).

[49] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300* (2024).

[50] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*. PMLR, 31094–31116.

[51] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314* (2024).

[52] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. 2024. Powerinfer: Fast large language model serving with a consumer-grade gpu. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 590–606.

[53] Biao Sun, Shuxin Zhang, et al. 2024. Llumnix: Dynamic Scheduling for Large Language Model Serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. https://www.usenix.org/system/files/osdi24-sun-biao.pdf

[54] DeepSeek Team et al. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. https://arxiv.org/abs/2501.12948. arXiv:2501.12948 [cs.CL].

[55] Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving math word problems with process-and outcome-based feedback. *arXiv preprint arXiv:2211.14275* (2022).

[56] Peiyi Wang, Lei Li, Zhihong Shao, RX Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. 2023. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. *arXiv preprint arXiv:2312.08935* (2023).

[57] Teng Wang, Zhangyi Jiang, Zhenqi He, Wenhan Yang, Yanan Zheng, Zeyu Li, Zifan He, Shenyang Tong, and Hailei Gong. 2025. Towards Hierarchical Multi-Step Reward Models for Enhanced Reasoning in Large Language Models. *arXiv preprint arXiv:2503.13551* (2025).

[58] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171* (2022).

[59] Zili Wang, Tianyu Zhang, Haoli Bai, Lu Hou, Xianzhi Yu, Wulong Liu, Shiming Xiang, and Lei Zhu. 2025. Faster and Better LLMs via Latency-Aware Test-Time Scaling. *arXiv preprint arXiv:2505.19634* (2025).

[60] Xinming Wei, Jiahao Zhang, Haoran Li, Jiayu Chen, Rui Qu, Maoliang Li, Xiang Chen, and Guojie Luo. 2025. Agent. xpu: Efficient Scheduling of Agentic LLM Workloads on Heterogeneous SoC. *arXiv preprint arXiv:2506.24045* (2025).

[61] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. LoongServe: Efficiently Serving Long-Context Large Language Models with Elastic Sequence Parallelism. In *Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP '24)*. doi:10.1145/3694715.3695948

[62] Yangzhen Wu, Zhiqing Sun, Shanda Li, Sean Welleck, and Yiming Yang. 2024. Inference scaling laws: An empirical analysis of compute-optimal inference for problem-solving with language models. *arXiv preprint arXiv:2408.00724* (2024).

[63] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

[64] An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. 2024. Qwen2. 5-math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122* (2024).

[65] Ling Yang, Zhaochen Yu, Bin Cui, and Mengdi Wang. 2025. Reasonflux: Hierarchical llm reasoning via scaling thought templates. *arXiv preprint arXiv:2502.06772* (2025).

[66] Zhongzhi Yu, Zheng Wang, Yuhan Li, Ruijie Gao, Xiaoya Zhou, Sreenidhi Reddy Bommu, Yang Zhao, and Yingyan Lin. 2024. Edge-llm: Enabling efficient large language model adaptation on edge devices via unified compression and adaptive layer voting. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.

[67] Liangqi Yuan, Dong-Jun Han, Shiqiang Wang, and Christopher G Brinton. 2025. Local-Cloud Inference Offloading for LLMs in Multi-Modal, Multi-Task, Multi-Dialogue Settings. *arXiv preprint arXiv:2502.11007* (2025).

[68] Qiyuan Zhang, Fuyuan Lyu, Zexu Sun, Lei Wang, Weixu Zhang, Wenyue Hua, Haolun Wu, Zhihan Guo, Yufei Wang, Niklas Muennighoff, et al. 2025. A Survey on Test-Time Scaling in Large Language Models: What, How, Where, and How Well? *arXiv preprint arXiv:2503.24235* (2025).

[69] Zhihao Zhang, Alan Zhu, Lijie Yang, Yihua Xu, Lanting Li, Phitchaya Mangpo Phothilimthana, and Zhihao Jia. 2024. Accelerating retrieval-augmented language model serving with speculation. *arXiv preprint arXiv:2401.14021* (2024).

[70] Jian Zhao, Runze Liu, Kaiyan Zhang, Zhimu Zhou, Junqi Gao, Dong Li, Jiafei Lyu, Zhouyi Qian, Biqing Qi, Xiu Li, et al. 2025. Genprm: Scaling test-time compute of process reward models via generative reasoning. *arXiv preprint arXiv:2504.00891* (2025).

[71] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems* 37 (2024), 62557–62583.

[72] Yue Zheng, Yuhao Chen, Bin Qian, Xiufang Shi, Yuanchao Shu, and Jiming Chen. 2025. A review on edge large language models: Design, execution, and applications. *Comput. Surveys* 57, 8 (2025), 1–35.

[73] Zhen Zheng, Xin Ji, Taosong Fang, Fanghao Zhou, Chuanjie Liu, and Gang Peng. 2024. Batchllm: Optimizing large batched llm inference with global prefix sharing and throughput-oriented token batching. *arXiv preprint arXiv:2412.03594* (2024).

[74] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. https://www.usenix.org/system/files/osdi24-zhong-yinmin.pdf

A Proof of Optimality for Prefix-Aware Scheduling

This appendix provides the formal proof that the greedy scheduling algorithm is optimal under a simplified set of assumptions that reflect a highly memory-constrained environment.

A.1 Assumptions

We make the following list of assumptions to facilitate our formulation and analysis in Sec. 4.2.

1. **Constant Total Work**: For a given set of CoTs to be scheduled in a single TTS iteration, the total number of unique beams (nodes) is fixed. This allows the problem of minimizing eviction cost, $\sum(\text{Nodes}(T_i) - P(T_i, T_{i+1}))$, to be simplified to maximizing the shared prefix sum, $\sum P(T_i, T_{i+1})$.
2. **No Preemption During Execution**: The schedule is determined once per TTS iteration, and the execution of the CoTs is non-preemptive.
3. **Homogeneous Generation Length**: The number of tokens generated for each beam within a single scheduling cycle is uniform.

A.2 Proof of Local Optimality

Assumptions. **Single CoT Batches**: The KV cache has a limited capacity such that only a single CoT can fit into memory at one time. This simplifies the problem by making each Trie, T_i, equivalent to a single CoT, c_i.

Objective. We prove that the greedy schedule, S_G, is locally optimal. A schedule is defined as locally optimal if its total score cannot be improved by a single swap of any two elements. The surrogate score for a schedule $S = (c_1, c_2, \ldots, c_L)$ is the sum of shared prefixes between consecutive elements as mentioned in Sec. 4.2:

$$\text{Score}(S) = \sum_{k=1}^{L-1} P(c_k, c_{k+1})$$

We will show that for a schedule S' created by swapping two elements in S_G, the change in score, $\Delta\text{Score} = \text{Score}(S') - \text{Score}(S_G)$, is non-positive ($\leq 0$).

The Greedy Invariant. The proof is based on the greedy invariant in Sec. 4.2. Formally:

$$P(c_{k-1}, c_k) = \max_{c_m \in Q} P(c_{k-1}, c_m)$$

The Interchange Argument. Consider the greedy schedule S_G and a new schedule S' created by swapping two elements, c_i and c_j, where $i < j - 1$. Our goal is to show that S' is no better than S_G.

- **Greedy Schedule (S_G):**
  $S_G = (\ldots, c_{i-1}, \mathrm{c_i}, c_{i+1}, \ldots, c_{j-1}, \mathrm{c_j}, c_{j+1}, \ldots)$
- **Swapped Schedule (S'):**
  $S' = (\ldots, c_{i-1}, \mathrm{c_j}, c_{i+1}, \ldots, c_{j-1}, \mathrm{c_i}, c_{j+1}, \ldots)$

The change in the total score is derived from the four connections affected by the swap.

$$\begin{aligned}\Delta\text{Score} = &\underbrace{\left[P(c_{i-1}, c_j) - P(c_{i-1}, c_i)\right]}_{\text{Term A}} \\ &+ \underbrace{\left[P(c_j, c_{i+1}) - P(c_i, c_{i+1})\right]}_{\text{Term B}} \\ &+ \underbrace{\left[P(c_{j-1}, c_i) - P(c_{j-1}, c_j)\right]}_{\text{Term C}} \\ &+ \underbrace{\left[P(c_i, c_{j+1}) - P(c_j, c_{j+1})\right]}_{\text{Term D}}\end{aligned}$$

To show that $\Delta\text{Score} \leq 0$, we demonstrate that each term in the expression is non-positive. We provide the argument for Term A; a symmetric argument holds for the remaining terms.

By the greedy invariant,

$$P(c_{i-1}, c_i) = \max_{c_m \in Q} P(c_{i-1}, c_m) \geq P(c_{i-1}, c_j)$$

Hence,

$$P(c_{i-1}, c_j) - P(c_{i-1}, c_i) \leq 0$$

Since all four terms are non-positive, their sum must also be non-positive. Therefore, $\Delta\text{Score} \leq 0$, and no single swap can improve the score.

B Artifact Appendix

B.1 Abstract

FastTTS is a framework for fast test-time scaling on edge devices, leveraging speculative beam extension, prefix caching, and adaptive memory allocation. The artifact includes the complete source code, benchmark scripts, and configuration files to reproduce the goodput, latency, and accuracy results presented in the paper. It supports evaluating AIME 2024 and AMC 2023 datasets using automated workflows.

B.2 Artifact check-list (meta-information)

- **Algorithm:** FastTTS
- **Program:** Python 3.11
- **Model:** Qwen2.5-Math-1.5B-Instruct, Qwen2.5-Math-7B-Instruct, Math-Shepherd-Mistral-7B-PRM, Skywork-o1-Open-PRM-Qwen-2.5-1.5B
- **Data set:** AIME 2024, AMC 2023 (via HuggingFace)
- **Run-time environment:** Linux, Conda, Python 3.11, CUDA 12.9, PyTorch 2.7.0
- **Hardware:** NVIDIA GeForce RTX 4090 GPU (24 GB VRAM)
- **Execution:** Automated via `run_all_experiments.py`
- **Metrics:** Goodput, Latency, Accuracy
- **Output:** JSONL logs, PDF plots
- **Experiments:** Verifier-guided Beam Search across datasets to reproduce goodput, latency, accuracy results.
- **How much disk space required (approximately)?:** ~200GB (including model weights and environment).
- **How much time is needed to prepare workflow (approximately)?:** ~15 minutes
- **How much time is needed to complete experiments (approximately)?:** ~60 GPU hours (depending on GPU)
- **Publicly available?:** No
- **Data licenses (if publicly available)?:** MIT/Apache 2.0 (Dataset dependent)
- **Workflow automation framework used?:** Custom Python scripts

B.3 Description

B.3.1 How to access. The artifact is available `here`.

B.3.2 Hardware dependencies. The experiments require a Linux machine with at least one NVIDIA GeForce RTX 4090 GPU (24 GB VRAM).

B.3.3 Software dependencies. Described in the code.

B.3.4 Data sets. The artifact uses the following datasets, automatically downloaded via the Hugging Face Datasets library:

- **AIME 2024**
- **AMC 2023**

B.3.5 Models. The following models are automatically downloaded from Hugging Face Hub:

- Generators:
  - `Qwen/Qwen2.5-Math-1.5B-Instruct`
  - `Qwen/Qwen2.5-Math-7B-Instruct`
- Verifiers:
  - `peiyi9979/math-shepherd-mistral-7b-prm`
  - `Skywork/Skywork-o1-Open-PRM-Qwen-2.5-1.5B`

B.4 Installation

For details, please read the *README.md* file in the repository.

1. Download the repository.
2. Create and activate the Conda environment:
```
conda env create -f environment.yml
conda activate FastTTS
```
3. Install the package in editable mode:
```
pip install -e .
```
4. Install the modified Skywork inference module:

```
cd modified-skywork-o1-prm-inference
pip install -e .
cd ..
```

B.5 Experiment workflow

The main results can be reproduced using:

```
run_all_experiments.py
```

To run all experiments (AIME and AMC datasets, multiple model sizes, varying N):

```
python run_all_experiments.py --exp
```

This command runs the benchmark configurations defined in `benchmarks/configs/`.

To generate the plots from the collected results:

```
python run_all_experiments.py --plot
```

To run both experiments and plotting in sequence:

```
python run_all_experiments.py --exp --plot
```

B.6 Evaluation and expected results

Upon successful execution, the script generates the following PDF figures in `benchmarks/benchmark_results/figs/`:

- `main_results_combined.pdf`: Compares Goodput across different methods and model sizes. It corresponds to Figure 12.
- `latency_combined.pdf`: Shows the latency breakdown. It corresponds to Figure 13.
- `acc.pdf`: Displays the accuracy metrics for the evaluated benchmarks. It corresponds to Figure 14a.

The results should match the trends reported in Figure 12, Figure 13, and Figure 14a of the paper, demonstrating the efficiency improvements.

B.7 Experiment customization

Experiments can be customized by modifying the YAML configuration files located in `benchmarks/configs/`. Individual benchmarks can be run via `benchmarks/run_benchmarks.py` with a specific config file.

B.8 Notes

Ensure that the `conda` environment is activated before running any scripts. The initial run may take longer due to model downloads.